BioSB Summer School on Advanced Machine Learning and AI for Life Sciences

Lecture 1 – Explainable AI (XAI): Feature Attribution (PDP, LIME, SHAP)

Dr. Pavel Sinitcyn[1,2]

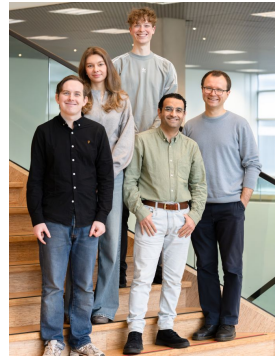
[1]Information and Computing Sciences
AI Technology for Life

[2]Pharmaceutical Sciences
Biomolecular Mass Spectrometry and Proteomics
Utrecht University

24 August 2026

Introduction

- Pavel Sinitcyn – p.sinitcyn@uu.nl – sinitcynlab.org
- Assistant Professor - Computational Biomolecular Mass Spectrometry Group
  - AI Technologies for Life Science (AIT4Life)
  - Biomolecular Mass Spectrometry and Proteomics (BioMS)
- Previously:
  - PostDoc - University of Wisconsin-Madison
  - PhD in Computational Mass Spectrometry - Max Planck Institute of Biochemistry
  - MSc in Bioinformatics - Lomonosov Moscow State University
- Looking for interns! Just write an email with your CV and motivation.

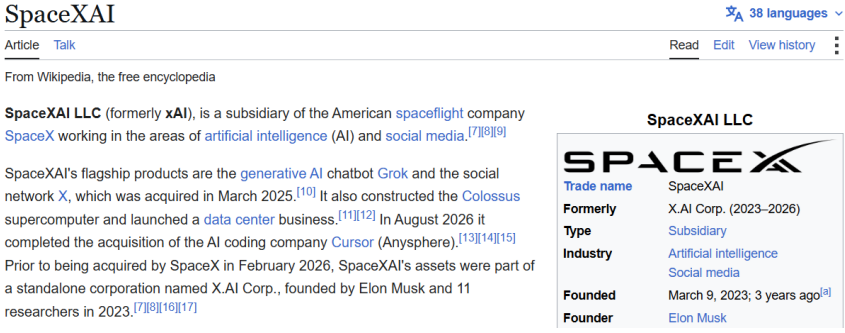


Lecture 1 – Plan

Utrecht University

The XAI disclaimer

We are not going to talk about xAI the company, for sure...



SpaceXAI

38 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

SpaceXAI LLC (formerly **xAI**), is a subsidiary of the American spaceflight company SpaceX working in the areas of artificial intelligence (AI) and social media.[7][8][9]

SpaceXAI's flagship products are the generative AI chatbot Grok and the social network X, which was acquired in March 2025.[10] It also constructed the Colossus supercomputer and launched a data center business.[11][12] In August 2026 it completed the acquisition of the AI coding company Cursor (Anysphere).[13][14][15] Prior to being acquired by SpaceX in February 2026, SpaceXAI's assets were part of a standalone corporation named X.AI Corp., founded by Elon Musk and 11 researchers in 2023.[7][8][16][17]

SpaceXAI LLC

SPACEX

Trade name	SpaceXAI
Formerly	X.AI Corp. (2023–2026)
Type	Subsidiary
Industry	Artificial intelligence Social media
Founded	March 9, 2023; 3 years ago[a]
Founder	Elon Musk

...though explainable AI (XAI) really is the rocket science of ML.

The husky and the wolf

Ribeiro et al. (2016) trained a classifier to tell huskies from wolves. On the test set it worked.

Then they asked: *why?*

The model was not looking at the animal. It was looking at the **snow in the background**.

The prediction was right. The reasoning was not.

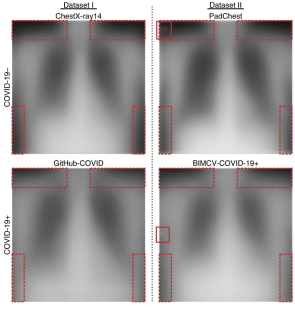
The whole motivation for XAI fits in this picture: accuracy alone is not enough.

(a) Husky classified as wolf (b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

Two more of the same story

- **COVID chest X-rays** (DeGrave et al., 2021). A model predicting COVID-19 from Chest X-Ray keyed on *hospital tags* and *patient positioning*, not lung pathology.
- **Dermatology rulers** (Bevan et al., 2022). Skin-cancer classifiers learnt that suspicious lesions are the ones doctors photograph *next to a ruler*.



COVID CXR – DeGrave et al. 2021

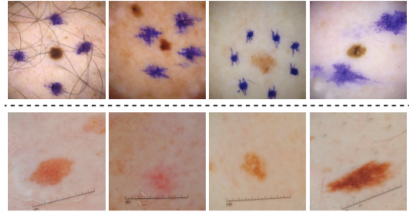


Figure 1: Examples of artefacts seen in ISIC 2020 data (Rotemberg et al., 2021). Top row shows images with surgical markings present, bottom row shows images with rulers.

Dermatology rulers – Bevan et al. 2022

The question we will keep asking

"The model is right —

but is it right *for the right reasons*?"

Every method today (PDP, LIME, SHAP) is a different way of answering this.

Black Boxes

A scoping choice for today

Within supervised learning we distinguish **classification** (y a class label) from **regression** (y a real number).

We will illustrate every method today on a **classification** example – a model predicting cancer risk from a small panel of biomarkers.

Why classification. The output is a single probability, which makes it easy to ask "what pushed this number up or down?" – the question every attribution method answers. All three methods transfer directly to regression.

Three practical axes

Utrecht University

We place every XAI method on three axes:

- **Intrinsic vs. post-hoc.** Is the model interpretable by design, or do we bolt on explanations afterwards?
- **Local vs. global.** Does the explanation describe *one prediction*, or the *whole model*?
- **Model-specific vs. model-agnostic.** Does the method peek inside the model, or treat it as a black box?

A fourth question is independent: **what does the explanation look like?**
Today: always a *feature score*.

Where PDP, LIME and SHAP sit

Method	**Intrinsic / post-hoc**	**Local / global**	**Specific / agnostic**
PDP	post-hoc	global	model-agnostic
LIME	post-hoc	local	model-agnostic
SHAP	post-hoc	local *and* global	agnostic *or* specific

All three treat the model as something we *query*, not something we *open up*. That is exactly what makes them useful on any model you will train in this course.

Methods disagree – and that is information

- Run PDP, LIME and SHAP on the same model. Compare their top features.
- You will rarely get full agreement. Each method makes different assumptions about how features interact and what "locality" or "contribution" means.
- Treat XAI output as a *hypothesis*, not a verdict – and if two methods with different assumptions agree, that is stronger evidence than either alone.

Explanations can even be adversarially attacked – Dombrowski et al., 2019

Today's roadmap

Utrecht University

1. **PDP** – how does the average prediction change as one feature varies? *(global)*
2. **LIME** – fit a simple model around one prediction. *(local, heuristic)*
3. **SHAP** – the same local idea, but derived from a fairness axiom in game theory *(local and global, principled)*.

Running example throughout: a model predicts **cancer risk** for **Patient 042** from Age, Gene A expression, TP53 mutation status, and Protein B level.

Next Section

Utrecht University

Transparent models vs. black boxes

Transparent models. A linear model's coefficients *are* the explanation: $w_1 = +0.72$ for TP53 mutation, $w_2 = +0.55$ for Gene A expression, and so on. A shallow decision tree can be read top to bottom.

Transparent by design – but predictive performance on complex, high-dimensional biological data is often limited.

Black-box models. Random forests, gradient boosting, neural networks: $f(x) = ?$

Accurate, but opaque. We cannot read off "why" from the parameters.

This is where PDP, LIME and SHAP come in – they query f from the outside.

Global vs. local explanation

Utrecht University

Global explanation

"What does the model rely on, in general?"

- Looks at behaviour across the whole dataset.
- Which features matter *on average*?
- Useful for model auditing and biomarker discovery.
- *PDP and SHAP lives here.*

Local explanation

"Why this prediction, for this sample?"

- Looks at one prediction at a time.
- Which features drove *this* output?
- Useful for individual decisions and trust.
- *LIME and SHAP live here.*

Most clinical questions are local. Most biomarker-discovery questions are global. We need both.

Running example: Patient 042

Utrecht University

Age	62
Gene A expression	high
Protein B (serum)	elevated
TP53 mutation	yes

Model f (a random forest) predicts: **high cancer risk**, $p = 0.87$.

What did the model use to reach this conclusion? Was it the mutation? The biomarkers? The age?

What is feature attribution?

Utrecht University

Feature attribution: assign a number (importance or contribution score) to each input feature, describing how much that feature influenced the model's output.

Feature	Contribution
TP53 mutation	$+0.32$
Gene A (high)	$+0.21$
Age (62)	$+0.12$
Protein B	$+0.08$
Blood pressure (normal)	-0.06

Partial Dependence Plot (PDP): the idea

Utrecht University

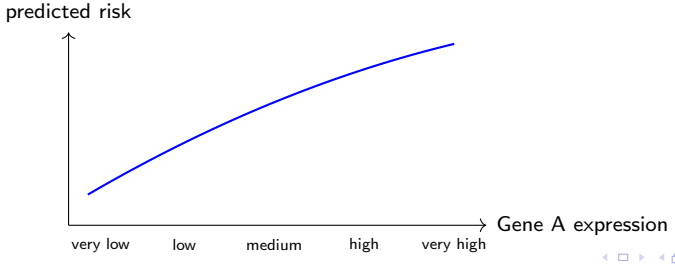
Question: how does the *average* model output change as one feature varies across its range?

1. Pick a feature x_j (e.g. Gene A expression) and a value v.
2. For *every* patient in the dataset: replace $x_j \to v$, keep all other features as-is.
3. Run the model on all n modified patients; average the outputs.
4. Repeat for every v across the range $\to$ connect the dots $\to$ the PDP curve.

PDP – the formula

$$\mathrm{PDP}_j(v) \;=\; \frac{1}{n}\sum_{i=1}^{n} f\left(v,\, x_{-j}^{(i)}\right)$$

- $x_{-j}^{(i)}$: all features of patient i *except* j, left untouched.
- **Freeze** $x_j = v$ for every patient in the dataset.
- **Average** the n model outputs $\rightarrow$ one point on the curve.
- **Sweep** v across the observed range of x_j $\rightarrow$ the full PDP curve.



Worked example: PDP(Gene A = high)

We sweep Gene A across its range; here is the computation for the single value $v = \text{high}$.

Sample	Gene A (x_j)	Age	TP53	Protein B	$f(\cdot)$
1	HIGH	62	yes	2.1	0.71
2	HIGH	45	no	0.8	0.48
3	HIGH	58	yes	1.9	0.69
4	HIGH	71	no	3.2	0.55
⋮	HIGH	⋮	⋮	⋮	⋮

$$\text{PDP}(\text{Gene A} = \text{high}) = \frac{0.71 + 0.48 + 0.69 + 0.55 + \cdots}{500} \approx 0.61$$

Orange = frozen at $v = \text{high}$; the rest of each row keeps the patient's real values.

Two things to watch for with PDP

1. Averaging hides heterogeneity.
If half the patients' risk goes up with Gene A and half goes down, the average curve can be flat – looking like "no effect" when the truth is "two very different effects".

Fix: **ICE** *(Individual Conditional Expectation)* – the same computation, but one curve *per patient*, no averaging. PDP is just the mean of all ICE curves.

2. Correlated features → extrapolation.
PDP freezes Gene A at "high" but keeps each patient's *original* Protein B – including combinations that never occur in real biology.

Fix: **ALE** *(Accumulated Local Effects)* – only compares patients *locally*, so it never leaves the real data distribution.

Today: names only – both are one line of code away in any XAI library once you understand PDP. You will find optional tasks in Assignments to try it our in practise.

From global surrogates to local surrogates

A **surrogate model** mimics a complex model with a simpler, interpretable one.

Global surrogate. Train one interpretable model (tree, linear) to match f everywhere. Rarely works – a deep model is not well approximated globally by a shallow tree.

Local surrogate (LIME). Approximate f only *near one patient*.

1. Perturb x to create a local neighbourhood.
2. Query the black box on those neighbours.
3. Fit a simple, weighted model on the neighbourhood.
4. Read off the local feature contributions.

Key idea: never try to explain f everywhere – only where it matters for *this* prediction.

What does LIME stand for?

Utrecht University

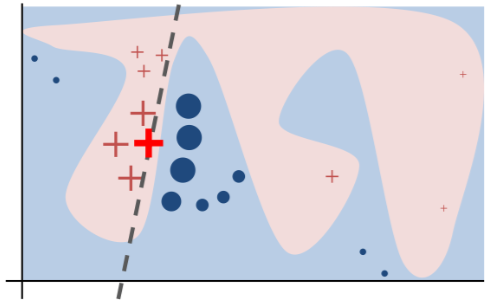
Local **I**nterpretable **M**odel-agnostic **E**xplanations – Ribeiro, Singh & Guestrin, 2016

Local	Explains one prediction: the behaviour of f near one sample x.
Interpretable	Uses a simple model humans can read – usually a sparse linear model.
Model-agnostic	Only needs input-output queries to f. No internals required.
Explanations	Returns a compact list of influential features.

LIME is not a global explanation. It builds a simple explanation *around the case we care about*.

LIME: local surrogate, visually

A simple line can mimic f well *near* x, even if it is wrong far away. LIME only cares about points close to the sample being explained.



Toy example to present intuition for LIME – Ribeiro et al., 2016

Two criteria for a good explanation

LIME explicitly optimises both – you cannot have one without the other.

(1) Fidelity $L(f, g, \pi_x)$: how well does g match f near x?

$$L(f, g, \pi_x) = \sum_z \pi_x(z) \left[f(z) - g(z) \right]^2$$

Poor fidelity $\Rightarrow$ g gives wrong predictions near x $\Rightarrow$ the explanation is misleading.

(2) Complexity $\Omega(g)$: how simple is g? (e.g. number of non-zero weights)
Low complexity: few features, human-readable. High complexity defeats the purpose.

Trade-off: perfect fidelity can be too complex; zero complexity is useless. LIME finds the sweet spot.

The LIME objective

$$\text{explanation}(x) \;=\; \underset{g \in G}{\operatorname{argmin}} \;\; L(f, g, \pi_x) \;+\; \Omega(g)$$

- $L(f, g, \pi_x)$: fidelity loss – keep g close to f near x.
- $\Omega(g)$: complexity penalty – keep g small and readable.

If g is linear (LIME's default):

$$g(z) = w_0 + w_1 z_1 + \cdots + w_k z_k$$

$\Omega(g)$ prefers few non-zero coefficients. LIME enforces this with **L1 (Lasso) regularisation**, which pushes most coefficients exactly to zero – the analyst sets K, the max number of features shown.

How LIME works: four steps

Utrecht University

1. **Create a neighbourhood.** Randomly perturb x – for tabular data, add noise to each feature. Generate $N \approx 1000$ perturbed copies z.
2. **Query the black box.** Feed every z into f; record $f(z)$. No model internals needed.
3. **Weight by proximity.** Compute a weight $\pi_x(z)$ for each z – closer to x means higher weight (Gaussian kernel).
4. **Fit a sparse local model.** Train a weighted, L1-regularised linear model g on the neighbourhood: $f(z) \approx g(z)$ near x. The coefficients *are* the feature attributions.

LIME, step by step: Patient 042

x (Patient 042)	Perturb ($N \approx 1000$)	Query $f(z)$	Weight $\pi_x(z)$	Fit $g(z)$ (sparse)
Gene A = high	Gene A = low, TP53=yes,...	0.81	0.91 (close)	TP53 +0.32
TP53 = yes	Gene A = med, TP53=no,...	0.44	0.78 (close)	Gene A +0.21
Age = 62	...	0.67	0.34 (mid)	Age +0.12
...	...	0.29	0.05 (far)	...

Attribution: TP53 mutation $+0.32$, Gene A expression $+0.21$, Age $+0.12$, Blood pressure -0.06.

Exactly the table we drew earlier by hand – now we know how LIME produces it.

The last bit - proximity weight $\pi_x(z)$

Utrecht University

Closer neighbours teach the local model more than distant ones.

$$\pi_x(z) = \exp\left(-\frac{d(x,z)^2}{2\sigma^2}\right)$$

- $d(x,z)$: distance between the original sample x and the perturbed neighbour z.
- σ: kernel width – controls what "local" means.

Samples close to x represent the model's true local behaviour; far-away samples may reflect completely different behaviour and should count for little.

Limitations of LIME

- **Stability.** Perturbations are random. Rerun LIME $\rightarrow$ a slightly different fit, especially for less dominant features. Two runs can swap the #2 and #3 feature.
- **Neighbourhood width σ.** Too small $\rightarrow$ noisy, unstable surrogate. Too large $\rightarrow$ no longer local, g captures average behaviour instead.
- **Local fidelity.** If g fits f poorly near x, the coefficients explain *the surrogate*, not the model – always check a fidelity score (e.g. weighted R^2).

LIME's weights and kernel are *heuristic choices* the analyst makes. This raises a natural question: **is there a principled, unique way to assign feature contributions?** That question has an answer – from 1953, from game theory.

Next Section

Cooperative games: the setup

Utrecht University

Bear with me and forget machine learning for a moment – just players, payoffs, and fairness.

Players. A finite set $N = \{1, \ldots, n\}$. Any subset $S \subseteq N$ is a **coalition** (there are 2^n of them, including $\emptyset$ and the **grand coalition** N).

Value function. $v : 2^N \to \mathbb{R}$ scores every coalition – what that team is worth. By convention $v(\emptyset) = 0$; $v(N)$ is the total payoff to be split.

Notation reminder. For $N = \{1, 2, 3\}$: $|\{1, 3\}| = 2$, and a *permutation* is just an ordering of the players ($3! = 6$ orderings of three players).

Running example: three researchers writing a paper

Utrecht University

- **A** Alice – theory
- **B** Bob – experiments
- **C** Charlie – writing

They submit a paper that earns a $9,000 prize.

How should they split it? Equal split? By seniority? By size of contribution? – what does "fair" even mean?

The value function for our example

Imagine a parallel universe for each possible team – what would they have achieved alone or together?

Coalition S	Members	$v(S)$
$\emptyset$	–	0
$\{A\}$	Alice alone	1,000
$\{B\}$	Bob alone	2,000
$\{C\}$	Charlie alone	1,000
$\{A, B\}$	Alice + Bob	5,000
$\{A, C\}$	Alice + Charlie	4,000
$\{B, C\}$	Bob + Charlie	4,000
$\{A, B, C\}$	all three (grand)	9,000

Bob is the strongest alone; every pair earns *more* than the sum of its parts (synergy); teaming up all three pays best.

Goal: a fair payoff vector

A solution to a cooperative game assigns each player a payoff:

$$\phi = (\phi_1, \phi_2, \ldots, \phi_n)$$

- ϕ_i: the payoff assigned to player i.
- **Efficiency**: $\sum_i \phi_i = v(N)$. Nothing wasted, nothing invented – the whole \$9,000 gets split, not \$8,500 and not \$9,500.

We need a rule that turns the value-function table into three numbers ϕ_A, ϕ_B, ϕ_C that add up to \$9,000. What rule is *fair*?

Step 1: marginal contribution

The **marginal contribution** of player i to coalition S is the extra value gained when i joins S:

$$\Delta_i(S) = v(S \cup \{i\}) - v(S)$$

Example. How much does Alice add to {Bob}?

$$\Delta_A(\{B\}) = v(\{A, B\}) - v(\{B\}) = 5{,}000 - 2{,}000 = \$3{,}000$$

Alice's contribution *depends on who she joins* – we will need her marginal contribution to *every* coalition that does not yet contain her.

A first, failed idea: just sum the marginals

For each player, sum their marginal contribution over every coalition they could join.

Alice (4 coalitions she can join, out of $\{\emptyset, \{B\}, \{C\}, \{B, C\}\}$):

$$\Delta_A(\emptyset) = v(\{A\}) - v(\emptyset) = 1{,}000 - 0 = 1{,}000$$
$$\Delta_A(\{B\}) = v(\{A, B\}) - v(\{B\}) = 5{,}000 - 2{,}000 = 3{,}000$$
$$\Delta_A(\{C\}) = v(\{A, C\}) - v(\{C\}) = 4{,}000 - 1{,}000 = 3{,}000$$
$$\Delta_A(\{B, C\}) = v(\{A, B, C\}) - v(\{B, C\}) = 9{,}000 - 4{,}000 = 5{,}000$$

Sum for Alice alone: **\$12,000** – already more than the whole \$9,000 prize!

Doing this for Bob and Charlie too would overshoot the grand coalition value by a lot.
We need to weight each coalition, not just sum.

Four fairness axioms

Axiom	Statement	Moto
Efficiency	$\sum_i \phi_i(v) = v(N)$	split the whole pie
Symmetry	equal marginal contributions $\Rightarrow$ equal payoff	no favouritism
Null player	contributes nothing $\Rightarrow$ gets nothing	no free lunch
Linearity	$\phi_i(\alpha v + \beta u) = \alpha\phi_i(v) + \beta\phi_i(u)$	explanations of a sum add up

Theorem (Shapley, 1953). Exactly *one* payoff rule satisfies all four axioms at once. Reject an axiom, and other rules become reasonable – but accept them, and the answer is unique.

The Shapley value formula

$$\phi_i(v) \;=\; \sum_{S \subseteq N \setminus \{i\}} w(|S|, n)\,\big[v(S \cup \{i\}) - v(S)\big], \qquad w(|S|, n) = \frac{|S|!\,(n - |S| - 1)!}{n!}$$

How to read it. Look at every coalition S that does not yet contain i; compute i's marginal contribution to it; weight by coalition size; sum.

Weights for $n = 3$:

$\lvert S\rvert = k$	$w(k,3) = k!(2-k)!/3!$	value
0	$0! \cdot 2!/3!$	$1/3$
1	$1! \cdot 1!/3!$	$1/6$
2	$2! \cdot 0!/3!$	$1/3$

Demo: Alice's Shapley value, worked

S (excl. A)	$\lvert S\rvert$	$\Delta_A(S)$	weight	weight $\times\Delta$
$\emptyset$	0	$1{,}000 - 0 = 1{,}000$	$1/3$	333.3
$\{B\}$	1	$5{,}000 - 2{,}000 = 3{,}000$	$1/6$	500.0
$\{C\}$	1	$4{,}000 - 1{,}000 = 3{,}000$	$1/6$	500.0
$\{B, C\}$	2	$9{,}000 - 4{,}000 = 5{,}000$	$1/3$	1666.7
			$\phi_A =$	**3,000**

Every marginal contribution counted once, weighted by how likely that ordering is to occur – that is the whole idea.

Exercise: compute ϕ_B or/and ϕ_C by hand

Utrecht University

The Shapley value formula

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} w(|S|, n) \left[v(S \cup \{i\}) - v(S) \right], \qquad w(|S|, n) = \frac{|S|!\,(n - |S| - 1)!}{n!}$$

Game

Coalition S	Members	$v(S)$
$\emptyset$	–	0
$\{A\}$	Alice alone	1,000
$\{B\}$	Bob alone	2,000
$\{C\}$	Charlie alone	1,000
$\{A, B\}$	Alice + Bob	5,000
$\{A, C\}$	Alice + Charlie	4,000
$\{B, C\}$	Bob + Charlie	4,000
$\{A, B, C\}$	all three (grand)	9,000

Solution: ϕ_B and ϕ_C

Utrecht University

Bob (S excl. B):

S	$\Delta_B(S)$	w	$w\Delta$
$\emptyset$	2,000	1/3	666.7
$\{A\}$	$5000 - 1000 = 4{,}000$	1/6	666.7
$\{C\}$	$4000 - 1000 = 3{,}000$	1/6	500.0
$\{A, C\}$	$9000 - 4000 = 5{,}000$	1/3	1666.7
		$\phi_B =$	**3,500**

Charlie (S excl. C):

S	$\Delta_C(S)$	w	$w\Delta$
$\emptyset$	1,000	1/3	333.3
$\{A\}$	$4000 - 1000 = 3{,}000$	1/6	500.0
$\{B\}$	$4000 - 2000 = 2{,}000$	1/6	333.3
$\{A, B\}$	$9000 - 5000 = 4{,}000$	1/3	1333.3
		$\phi_C =$	**2,500**

Check: $\phi_A + \phi_B + \phi_C = 3{,}000 + 3{,}500 + 2{,}500 = \$9{,}000 = v(\{A, B, C\})$ ✓

Recap: cooperative games

Utrecht University

- **Cooperative game:** players N and a value function v on coalitions.
- **Marginal contribution:** $\Delta_i(S) = v(S \cup \{i\}) - v(S)$.
- **Shapley value:** the *unique* weighted average of marginal contributions satisfying efficiency, symmetry, null player, and linearity.

Now: how do we turn a machine-learning model into a cooperative game?

The key idea: ML as a cooperative game

Utrecht University

Treat features as players, the prediction as the payoff.

Cooperative game		**Model explainability**
Players $N = \{1, \ldots, n\}$	$\leftrightarrow$	features of one input $\{x_1, \ldots, x_n\}$
Coalition $S \subseteq N$	$\leftrightarrow$	subset of features that are "present"
Value function $v(S)$	$\leftrightarrow$	model output using only features in S
Shapley value ϕ_i	$\leftrightarrow$	attribution of feature i to *this* prediction

For Patient 042: $N = \{\text{Age, Gene A, TP53, Protein B}\}$, and ϕ_i is exactly the number we want in our attribution table – but now with a fairness guarantee behind it.

Two challenges in adopting Shapley for ML

Utrecht University

In the researcher example, v was handed to us. In ML, it is not.

Challenge I – defining $v(S)$. What does it mean for the model to use "only the features in S"? The model expects *all* of them as input.

Challenge II – computing v and ϕ. Even once v is defined, there are 2^n coalitions to evaluate – explosive growth.

Challenge I: what does $v(S)$ mean?

Utrecht University

Idea: instead of removing features, replace the missing ones with values that "represent their absence".

Marginal (interventional) value function.

$$v(S) \approx \frac{1}{|D|} \sum_{\text{background rows}} f\left(x_S,\ X_{-S}^{\text{background}}\right)$$

Freeze the features in S at Patient 042's values; for the rest, plug in values from a **background dataset** D (e.g. 100 training rows) and average the model output.

+ easy to estimate. − ignores feature dependencies (can create unrealistic combinations, just like PDP).

Marginal value function: mechanics

Computing $v(\{\text{Age}, \text{TP53}\})$ for Patient 042 (Age= 62, TP53=yes) – Gene A and Protein B come from the background set.

	Age	Gene A	TP53	Protein B	
Patient 042 (frozen)	62	?	yes	?	
Background row 1	62	low	yes	elevated	$\to f = 0.71$
Background row 2	62	high	yes	normal	$\to f = 0.66$
Background row 3	62	med	yes	elevated	$\to f = 0.79$

$$v(\{\text{Age}, \text{TP53}\}) \approx \text{mean}(0.71, 0.66, 0.79) \approx 0.72$$

Challenge II: computational cost

Even with $v(S)$ defined, the Shapley formula sums over *all* 2^n subsets, and each $v(S)$ costs $|D|$ model calls.

Features n	Coalitions 2^n	Comment
10	1,024	trivial
20	$\approx 10^6$	doable
30	$\approx 10^9$	already painful
100	$\approx 10^{30}$	impossible to enumerate

For $n = 30$, $|D| = 100$: naive cost $\approx 10^{11}$ model calls. **We need approximations.** That is what the SHAP family is.

Two strategies for tractable Shapley values

Utrecht University

Model-agnostic. Reformulate ϕ as the solution of a weighted linear regression, or estimate it by sampling coalitions.

$\rightarrow$ **KernelSHAP** (regression-based); also PermutationSHAP, SamplingSHAP (Monte-Carlo based).

Model-specific. Exploit the model's architecture for exact or much faster computation.

$\rightarrow$ **TreeSHAP** (trees, forests, boosting), **DeepSHAP** (neural networks) and **QuadraSHAP**.

Both trade some generality for speed.

Our lab development

Utrecht University

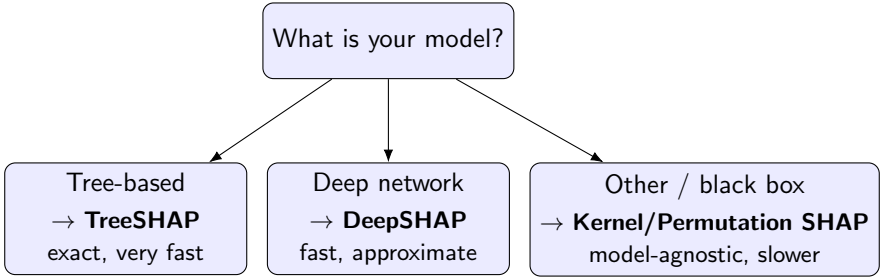
QuadraSHAP: Stable and Scalable Shapley Values for Product Games via Gauss–Legendre Quadrature

Majid Mohammadi[1,2] Grigory Reznikov[1,2] Pavel Sinitcyn[1,2]
Krikamol Muandet[3] Siu Lun Chau[4]

[1] AI Technology for Life, Information and Computing Sciences, Utrecht University, The Netherlands
[2] Biomolecular Mass Spectrometry and Proteomics, Pharmaceutical Sciences, Utrecht University, The Netherlands
[3]Rational Intelligence Lab, CISPA Helmholtz Center for Information Security, Germany
[4]Epistemic Intelligence & Computation Lab, College of Computing & Data Science, Nanyang Technological University, Singapore

Under revision in NeurIPS'26 – Mohammadi et al., 2026

Which SHAP variant should I use?



`shap.Explainer(model, X)` auto-selects the right algorithm for you.

Interpreting SHAP values in practice

SHAP values are signed, additive contributions – they tell you *direction* and *magnitude*:

$$f(x) = \underbrace{\mathbb{E}[f(x)]}_{\text{baseline}} + \sum_i \phi_i$$

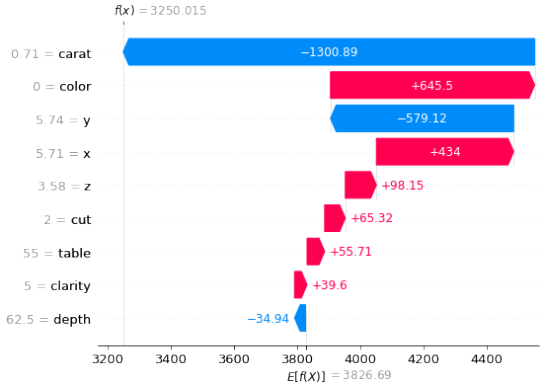
For Patient 042: baseline (average predicted risk) $= 0.20$, and

$$0.20 + \underbrace{(0.32 + 0.21 + 0.12 + 0.08 - 0.06)}_{=0.67} = 0.87 = f(x)$$

- $\phi_i > 0$: pushed the prediction *above* the baseline.
- $\phi_i < 0$: pushed it *below*.
- $|\phi_i|$: use for feature-importance rankings across the dataset.

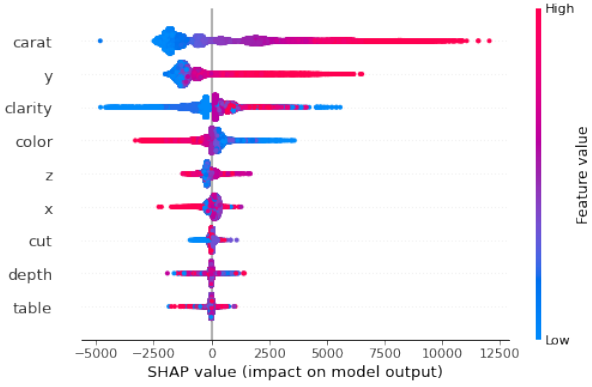
Local → global: one ϕ vector per patient (a force/waterfall plot); average $|\phi_i|$ across all patients recovers a global ranking (a beeswarm plot) – SHAP gives both for free.

Visualising one explanation



Waterfall plot from a SHAP explanation of an XGBoost model (TreeSHAP), predicting diamond prices – Kaggle. Patient 042's own waterfall plot would look just like this, with Age / Gene A / TP53 / Protein B on the axis instead.

Visualising all explanations



Reading the summary plot: features run down the left axis by descending importance, the horizontal axis shows SHAP value magnitude, and point colour (right axis) encodes each feature's actual value in the dataset.

Putting it together

Utrecht University

	PDP	LIME	SHAP
Scope	global	local	local *and* global
Basis	averaging	heuristic local fit	game-theoretic axioms
Guarantee	none	none	unique, fair (given axioms)
Cost	cheap	cheap	KernelSHAP slow, Tree/Deep fast

Accuracy is not enough.

Pick the tool that matches your question – and remember: *these are hypotheses about the model, not certainties about biology.*

References

Interpretable Machine Learning.
Molnar, C. (2022). https://christophm.github.io/interpretable-ml-book/

Ribeiro, M.T., Singh, S., Guestrin, C. *"Why should I trust you?" Explaining the predictions of any classifier.* KDD 2016.

Shapley, L.S. (1953). *A Value for n-Person Games.*

Lundberg, S.M., Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions.* NeurIPS 2017.

Lundberg, S.M., Erion, G., Lee, S.-I. (2018). *Consistent Individualized Feature Attribution for Tree Ensembles.* arXiv:1802.03888.

DeGrave, A.J., Janizek, J.D., Lee, S.-I. (2021). *AI for radiographic COVID-19 detection selects shortcuts over signal.* Nature Machine Intelligence 3.7: 610–619.

Bevan, P.J., Atapour-Abarghouei, A. (2022). *Skin deep unlearning: Artefact and instrument debiasing in the context of melanoma classification.* PMLR 162.